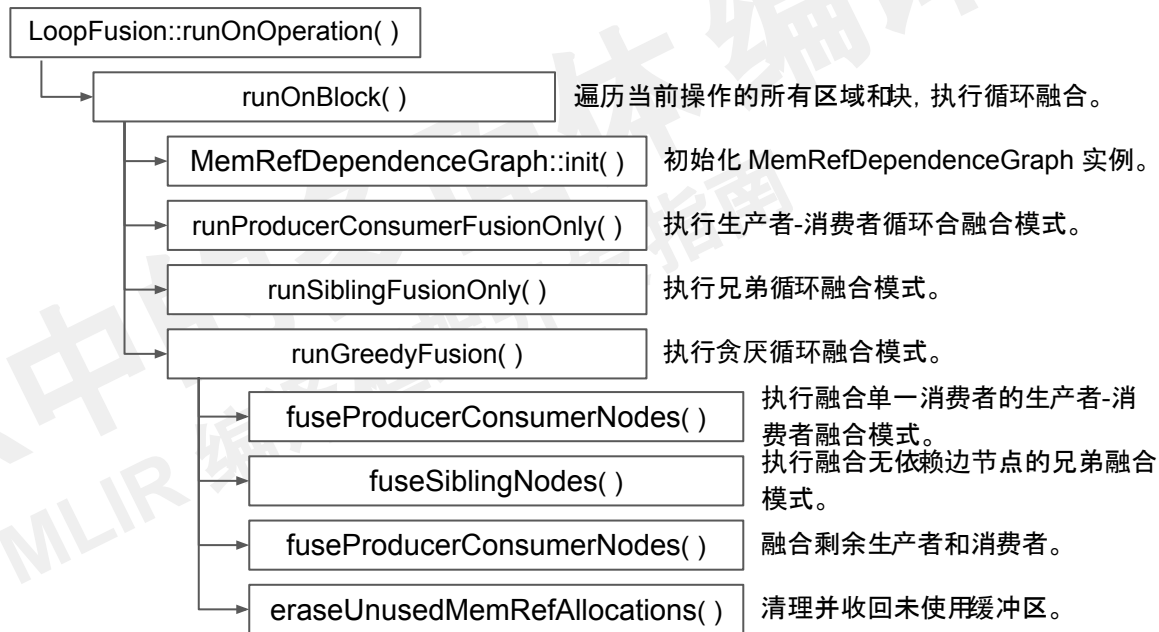


5.3.2. AffineLoopFusion pass 工作流程

LoopFusion::runOnOperation 函数调用过程

- (1) 构建内存依赖图
- (2) 执行生产者-消费者融合
- (3) 执行兄弟融合



生产者-消费者融合测试用例

```
func.func @producer_consumer_fusion(%arg0: memref<10xf32>, %arg1: memref<10xf32>) {  
  %0 = memref.alloc() : memref<10xf32>  
  %1 = memref.alloc() : memref<10xf32>  
  %cst = arith.constant 0.000000e+00 : f32  
  affine.for %arg2 = 0 to 10 {  
    affine.store %cst, %0[%arg2] : memref<10xf32>  
    affine.store %cst, %1[%arg2] : memref<10xf32>  
  }  
  affine.for %arg2 = 0 to 10 {  
    %2 = affine.load %0[%arg2] : memref<10xf32>  
    %3 = arith.addf %2, %2 : f32  
    affine.store %3, %arg0[%arg2] : memref<10xf32>  
  }  
  affine.for %arg2 = 0 to 10 {  
    %2 = affine.load %1[%arg2] : memref<10xf32>  
    %3 = arith.mulf %2, %2 : f32  
    affine.store %3, %arg1[%arg2] : memref<10xf32>  
  }  
  return  
}
```

构建内存依赖图 - 构造 nodes 和 memrefAccesses 集合

(1) 构建内存依赖图

(2) 执行生产者-消费者融合

(3) 执行兄弟融合

LoopFusion::runOnOperation()

runOnBlock()

MemRefDependenceGraph::init()

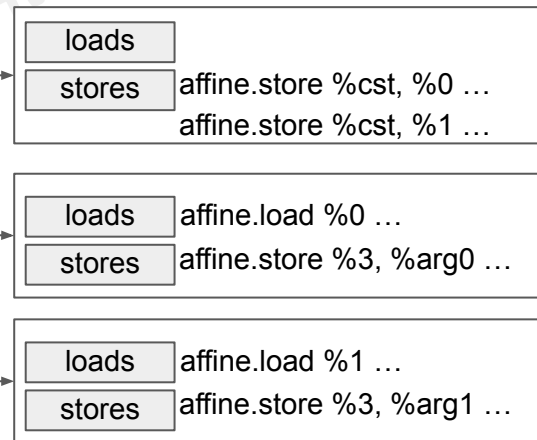
```
Node 0 %0 = memref.alloc() : memref<10xf32>
Node 1 %1 = memref.alloc() : memref<10xf32>
Node 2 %cst = arith.constant 0.000000e+00 : f32
Node 3 affine.for %arg2 = 0 to 10 {
affine.for 0  affine.store %cst, %0[%arg2] : memref<10xf32>
                affine.store %cst, %1[%arg2] : memref<10xf32>
            }
            affine.for %arg2 = 0 to 10 {
Node 4  %2 = affine.load %0[%arg2] : memref<10xf32>
affine.for 1  %3 = arith.addf %2, %2 : f32
                affine.store %3, %arg0[%arg2] : memref<10xf32>
            }
Node 5  affine.for %arg2 = 0 to 10 {
affine.for 2  %2 = affine.load %1[%arg2] : memref<10xf32>
                %3 = arith.mulf %2, %2 : f32
                affine.store %3, %arg1[%arg2] : memref<10xf32>
            }
            return
}
```

nodes 集合 (Node)

节点 id	节点实例
0	%0 = ...
1	%1 = ...
2	%cst = ...
3	affine.for 0
4	affine.for 1
5	affine.for 2

memrefAccesses 集合

memref	访问节点
%0	(3, 4)
%1	(3, 5)
%arg0	(4)
%arg1	(5)

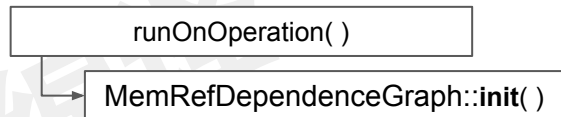


构建内存依赖图

(1) 构建内存依赖图

(2) 执行生产者-消费者融合

(3) 执行兄弟融合



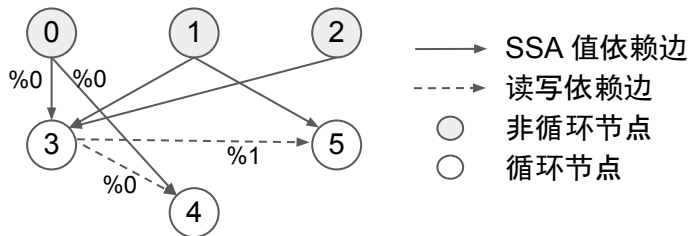
```
Node 0    %0 = memref.alloc() : memref<10xf32>
Node 1    %1 = memref.alloc() : memref<10xf32>
Node 2    %cst = arith.constant 0.000000e+00 : f32
Node 3    affine.for %arg2 = 0 to 10 {
affine.for 0    affine.store %cst, %0[%arg2] : memref<10xf32>
                affine.store %cst, %1[%arg2] : memref<10xf32>
            }
Node 4    affine.for %arg2 = 0 to 10 {
affine.for 1    %2 = affine.load %0[%arg2] : memref<10xf32>
                %3 = arith.addf %2, %2 : f32
                affine.store %3, %arg0[%arg2] : memref<10xf32>
            }
Node 5    affine.for %arg2 = 0 to 10 {
affine.for 2    %2 = affine.load %1[%arg2] : memref<10xf32>
                %3 = arith.mulf %2, %2 : f32
                affine.store %3, %arg1[%arg2] : memref<10xf32>
            }
            return
        }
```

nodes

节点 id	节点操作
0	%0 = ...
1	%1 = ...
2	%cst = ...
3	affine.for 0
4	affine.for 1
5	affine.for 2

memrefAccesses

memref	访问节点
%0	(3, 4)
%1	(3, 5)
%arg0	(4)
%arg1	(5)



执行生产者-消费者融合

- (1) 构建内存依赖图
- (2) 执行生产者-消费者融合
- (3) 执行兄弟融合

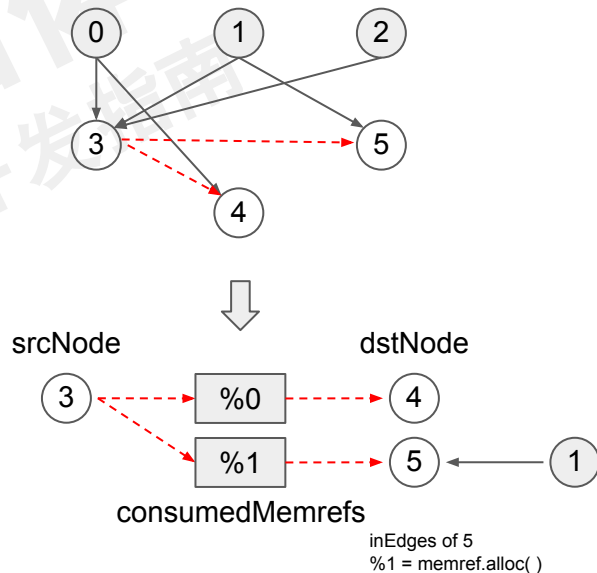
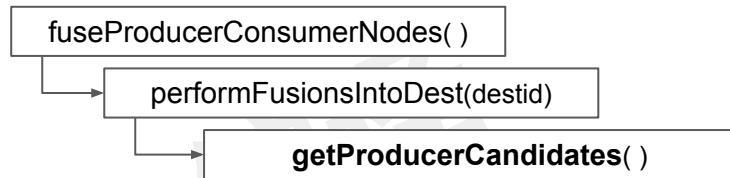
performFusionsIntoDest() 函数功能:

- 1) 获取候选源循环节点
- 2) 计算融合循环插入点
- 3) 判断生产者-消费者融合可行性
- 4) 计算循环融合收益
- 5) 执行循环融合

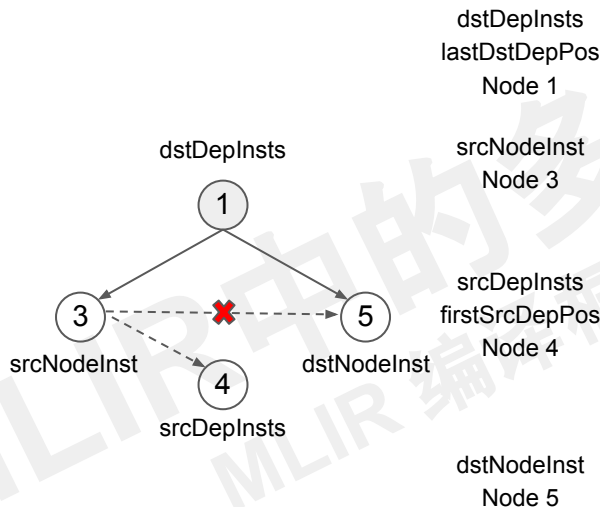
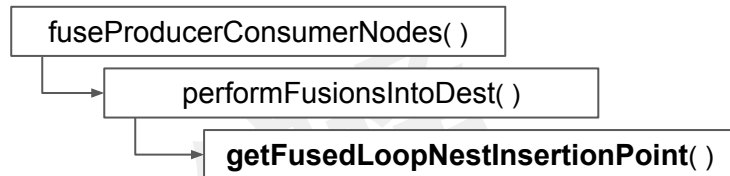


获取源循环候选节点

```
func.func @producer_consumer_fusion(%arg0:  
memref<10xf32>, %arg1: memref<10xf32>) {  
Node 0    %0 = memref.alloc() : memref<10xf32>  
Node 1    %1 = memref.alloc() : memref<10xf32>  
Node 2    %cst = arith.constant 0.000000e+00 : f32  
Node 3    affine.for %arg2 = 0 to 10 {  
affine.for 0    affine.store %cst, %0[%arg2] : memref<10xf32>  
                affine.store %cst, %1[%arg2] : memref<10xf32>  
    }  
    affine.for %arg2 = 0 to 10 {  
Node 4      %2 = affine.load %0[%arg2] : memref<10xf32>  
affine.for 1    %3 = arith.addf %2, %2 : f32  
                affine.store %3, %arg0[%arg2] : memref<10xf32>  
    }  
Node 5    affine.for %arg2 = 0 to 10 {  
affine.for 2    %2 = affine.load %1[%arg2] : memref<10xf32>  
                %3 = arith.mulf %2, %2 : f32  
                affine.store %3, %arg1[%arg2] : memref<10xf32>  
    }  
    return  
}
```



计算融合循环插入点



%1 = memref.alloc() : memref<10xf32>

写入 %1

affine.for %arg2 = 0 to 10 {

affine.store %cst, %0[%arg2] : memref<10xf32>

写入 %0

affine.store %cst, %1[%arg2] : memref<10xf32>

源循环节点

}

affine.for %arg2 = 0 to 10 {

%2 = affine.load %0[%arg2] : memref<10xf32>

← 插入点

读出 %0

%3 = arith.addf %2, %2 : f32

affine.store %3, %arg0[%arg2] : memref<10xf32>

}

affine.for %arg2 = 0 to 10 {

%2 = affine.load %1[%arg2] : memref<10xf32>

读出 %1

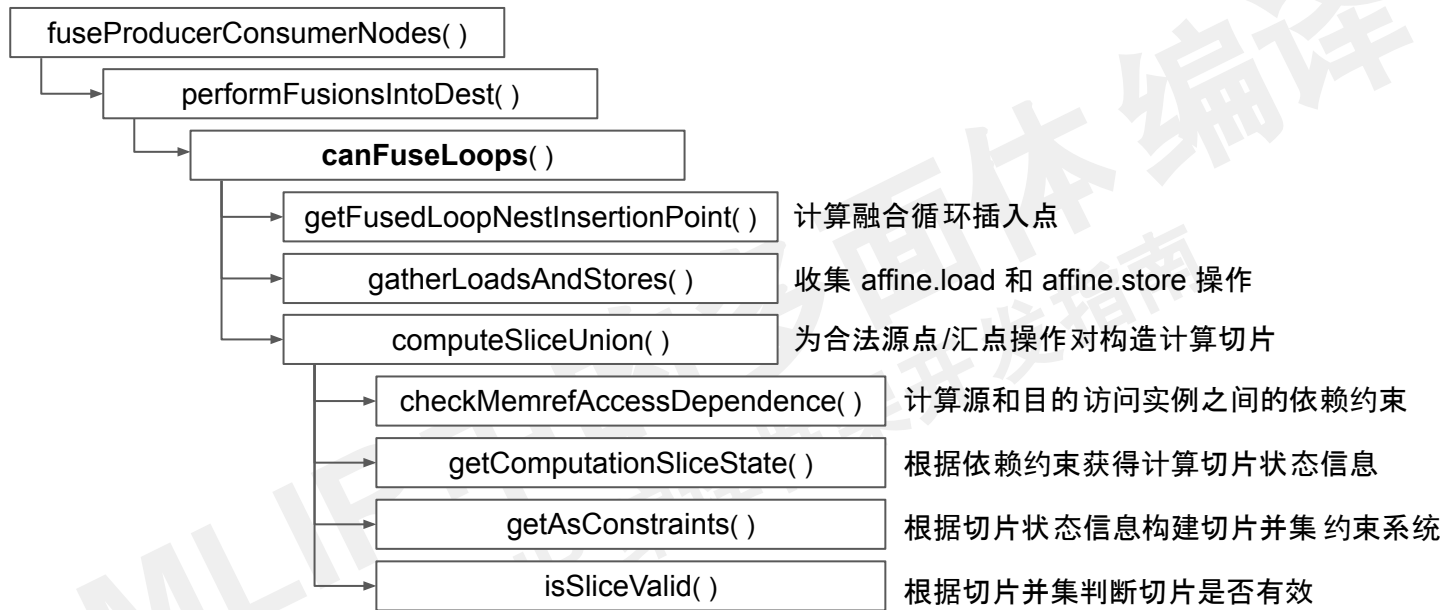
%3 = arith.mulf %2, %2 : f32

affine.store %3, %arg1[%arg2] : memref<10xf32>

}

目的循环节点

判断生产者-消费者融合可行性



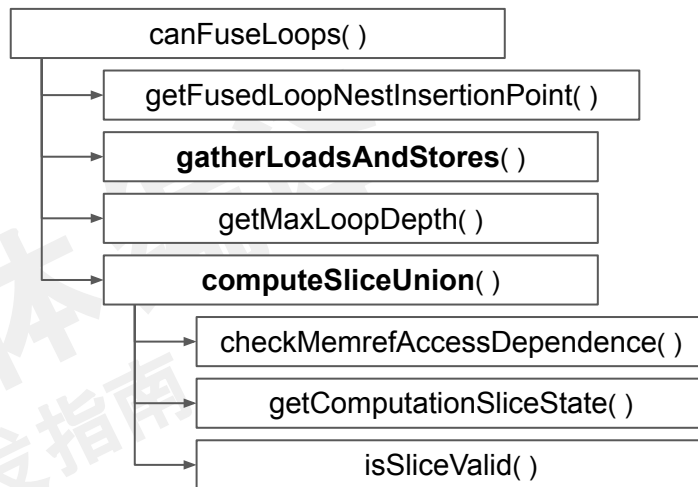
判断生产者-消费者融合可行性

```
func.func @producer_consumer_fusion(%arg0: memref<10xf32>,  
%arg1: memref<10xf32>) {  
  %0 = memref.alloc() : memref<10xf32>  
  %1 = memref.alloc() : memref<10xf32>  
  %cst = arith.constant 0.000000e+00 : f32  
  affine.for %arg2 = 0 to 10 {  
    affine.store %cst, %0[%arg2] : memref<10xf32>  
    affine.store %cst, %1[%arg2] : memref<10xf32>  
  }  
  affine.for %arg2 = 0 to 10 {  
    %2 = affine.load %0[%arg2] : memref<10xf32>  
    %3 = arith.addf %2, %2 : f32  
    affine.store %3, %arg0[%arg2] : memref<10xf32>  
  }  
  affine.for %arg2 = 0 to 10 {  
    %2 = affine.load %1[%arg2] : memref<10xf32>  
    %3 = arith.mulf %2, %2 : f32  
    affine.store %3, %arg1[%arg2] : memref<10xf32>  
  }  
  return  
}
```

Node 3
affine.for 0

Node 5
affine.for 2

读写依赖关系的 memref



生产者-消费者融合模式下 opsA 集合只收集 affine.store 操作

opsA

```
affine.store %cst, %0[%arg2] : memref<10xf32>  
affine.store %cst, %1[%arg2] : memref<10xf32>
```

opsB

```
%2 = affine.load %1[%arg2] : memref<10xf32>  
affine.store %3, %arg1[%arg2] : memref<10xf32>
```

判断生产者-消费者融合可行性

checkMemrefAccessDependence()

```
affine.for %arg2 = 0 to 10 {  
  affine.store %cst, %0[%arg2] : memref<10xf32>  
  affine.store %cst, %1[%arg2] : memref<10xf32>  
}  
...  
affine.for %arg2 = 0 to 10 {  
  %2 = affine.load %1[%arg2] : memref<10xf32>  
  %3 = arith.mulf %2, %2 : f32  
  affine.store %3, %arg1[%arg2] : memref<10xf32>  
}
```



dependenceConstraints

$$\begin{aligned} (-1) * \%arg2 + (1) * \%arg2' + 0 &= 0 \\ (1) * \%arg2 + (0) * \%arg2' + 0 &\geq 0 \\ (-1) * \%arg2 + (0) * \%arg2' + 9 &\geq 0 \\ (0) * \%arg2 + (1) * \%arg2' + 0 &\geq 0 \\ (0) * \%arg2 + (-1) * \%arg2' + 9 &\geq 0 \end{aligned}$$

getComputationSliceState()



sliceState/sliceUnion

ivs : { %arg2 }
lbs : { (d0) -> (d0) }
lbOperands : { %arg2' }
ubs : { (d0) -> (d0 + 1) }
ubOperands : { %arg2' }



```
affine.for %arg2 = 0 to 10 {  
  ...  
  affine.store %cst, %1[%arg2] : memref<10xf32>  
}  
affine.for %arg2 = 0 to 10 {  
  %2 = affine.load %1[%arg2] : memref<10xf32>  
  ...  
}
```

计算切片为最大切片

isSliceValid()

isSliceMaximalFastCheck()

canFuseLoops()

getFusedLoopNestInsertionPoint()

gatherLoadsAndStores()

getMaxLoopDepth()

computeSliceUnion()

checkMemrefAccessDependence()

getComputationSliceState()

isSliceValid()

```
affine.for %arg2' = 0 to 10 {  
  affine.for %arg2 = (d0) -> (d0)(%arg2') to  
    (d0) -> (d0 + 1)(%arg2') {  
    affine.store %cst, %0[%arg2] : memref<10xf32>  
    affine.store %cst, %1[%arg2] : memref<10xf32>  
  }  
  %2 = affine.load %1[%arg2'] : memref<10xf32>  
  %3 = arith.mulf %2, %2 : f32  
  affine.store %3, %arg1[%arg2'] : memref<10xf32>  
}
```

判断生产者-消费者融合可行性

checkMemrefAccessDependence()

```
affine.for %arg2 = 0 to 10 {  
  affine.store %cst, %0[%arg2] : memref<10xf32>  
  affine.store %cst, %1[%arg2] : memref<10xf32>  
}  
...  
affine.for %arg2 = 10 to 20 {  
  %2 = affine.load %1[%arg2] : memref<10xf32>  
  %3 = arith.mulf %2, %2 : f32  
  affine.store %3, %arg1[%arg2] : memref<10xf32>  
}
```



dependenceConstraints

```
( -1 ) * %arg2 + ( 1 ) * %arg2' + 0 = 0  
( 1 ) * %arg2 + ( 0 ) * %arg2' + 0 >= 0  
( -1 ) * %arg2 + ( 0 ) * %arg2' + 9 >= 0  
( 0 ) * %arg2 + ( 1 ) * %arg2' + (-10) >= 0  
( 0 ) * %arg2 + ( -1 ) * %arg2' + (19) >= 0
```

canFuseLoops()

getFusedLoopNestInsertionPoint()

gatherLoadsAndStores()

getMaxLoopDepth()

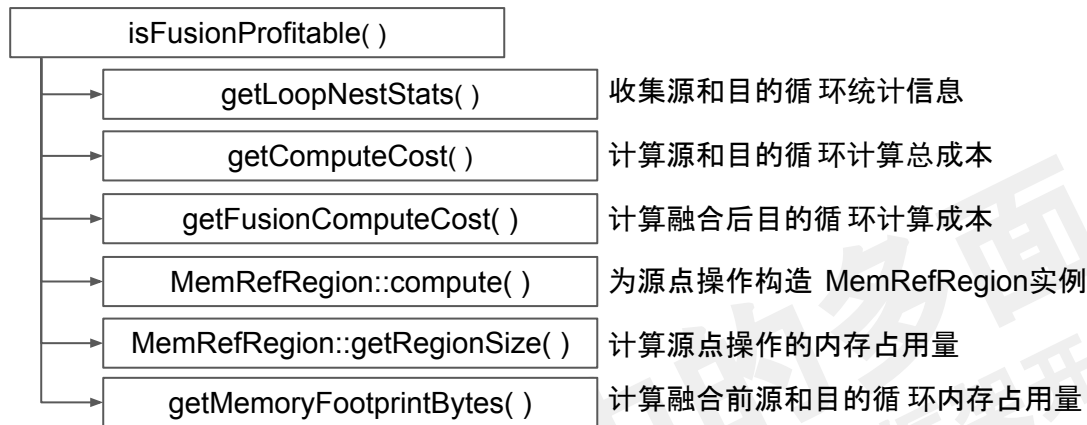
computeSliceUnion()

checkMemrefAccessDependence()

getComputationSliceState()

isSliceValid()

计算循环融合收益



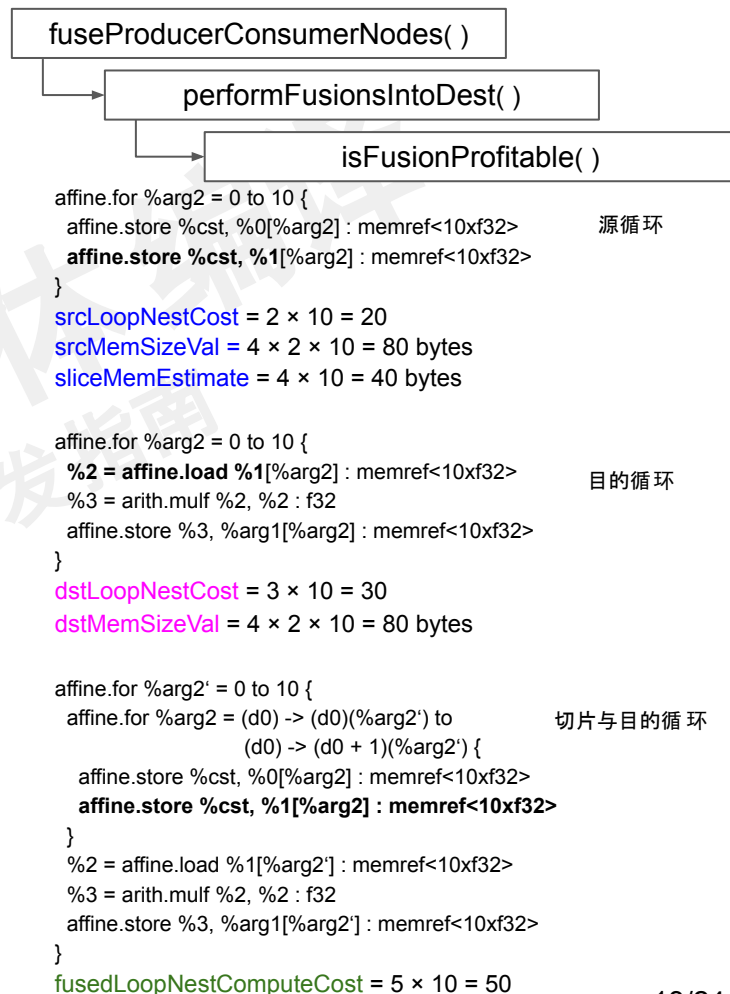
计算成本 = 操作数量 × 循环迭代总数

additionalComputeFraction =

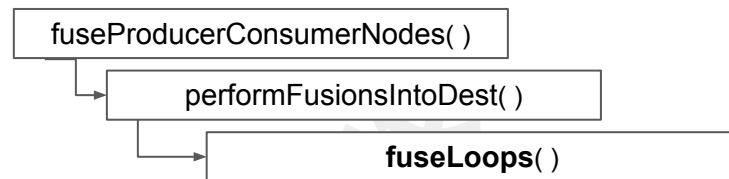
$$\text{fusedLoopNestComputeCost} / (\text{srcLoopNestCost} + \text{dstLoopNestCost}) - 1 \\ = 0 < 30\% (\text{computeToleranceThreshold})$$

融合前的内存占用量: $\text{srcMemSizeVal} + \text{dstMemSizeVal} = 80 + 80 = 160$

融合后的内存占用量: $\text{fusedMem} = \text{dstMemSizeVal} + \text{sliceMemEstimate} = 80 + 40 \\ = 120 < 160$



执行循环融合



```
func.func @producer_consumer_fusion(%arg0: memref<10xf32>,  
%arg1: memref<10xf32>) {  
  %0 = memref.alloc() : memref<10xf32>  
  %1 = memref.alloc() : memref<10xf32>  
  %cst = arith.constant 0.000000e+00 : f32  
  affine.for %arg2 = 0 to 10 {  
    affine.store %cst, %0[%arg2] : memref<10xf32>  
    affine.store %cst, %1[%arg2] : memref<10xf32>  
  }  
  affine.for %arg2 = 0 to 10 {  
    %2 = affine.load %0[%arg2] : memref<10xf32>  
    %3 = arith.addf %2, %2 : f32  
    affine.store %3, %arg0[%arg2] : memref<10xf32>  
  }  
  affine.for %arg2 = 0 to 10 {  
    %2 = affine.load %1[%arg2] : memref<10xf32>  
    %3 = arith.mulf %2, %2 : f32  
    affine.store %3, %arg1[%arg2] : memref<10xf32>  
  }  
  return  
}
```

循环融合



```
func.func @producer_consumer_fusion(%arg0: memref<10xf32>,  
%arg1: memref<10xf32>) {  
  %0 = memref.alloc() : memref<10xf32>  
  %1 = memref.alloc() : memref<10xf32>  
  %cst = arith.constant 0.000000e+00 : f32  
  affine.for %arg2 = 0 to 10 {  
    affine.store %cst, %0[%arg2] : memref<10xf32>  
    affine.store %cst, %1[%arg2] : memref<10xf32>  
    %2 = affine.load %1[%arg2] : memref<10xf32>  
    %3 = arith.mulf %2, %2 : f32  
    affine.store %3, %arg1[%arg2] : memref<10xf32>  
  }  
  affine.for %arg2 = 0 to 10 {  
    %2 = affine.load %0[%arg2] : memref<10xf32>  
    %3 = arith.addf %2, %2 : f32  
    affine.store %3, %arg0[%arg2] : memref<10xf32>  
  }  
  return  
}
```

执行循环化融合

```
func.func @producer_consumer_fusion(%arg0: memref<10xf32>,  
%arg1: memref<10xf32>) {  
  %0 = memref.alloc() : memref<10xf32>  
  %1 = memref.alloc() : memref<10xf32>  
  %cst = arith.constant 0.000000e+00 : f32  
  affine.for %arg2 = 0 to 10 {  
    affine.store %cst, %0[%arg2] : memref<10xf32>  
    affine.store %cst, %1[%arg2] : memref<10xf32>  
  }  
  affine.for %arg2 = 0 to 10 {  
    %2 = affine.load %0[%arg2] : memref<10xf32>  
    %3 = arith.addf %2, %2 : f32  
    affine.store %3, %arg0[%arg2] : memref<10xf32>  
  }  
  affine.for %arg2 = 0 to 10 {  
    %2 = affine.load %1[%arg2] : memref<10xf32>  
    %3 = arith.mulf %2, %2 : f32  
    affine.store %3, %arg1[%arg2] : memref<10xf32>  
  }  
  return  
}
```

⇒

```
affine.for %arg2 = (d0) -> (d0)(%arg2') to 10 {  
  affine.store %cst, %0[%arg2] : memref<10xf32>  
  affine.store %cst, %1[%arg2] : memref<10xf32>  
}
```

↓

```
affine.for %arg2 = (d0) -> (d0)(%arg2') to (d0) -> (d0 + 1)(%arg2') {  
  affine.store %cst, %0[%arg2] : memref<10xf32>  
  affine.store %cst, %1[%arg2] : memref<10xf32>  
}
```

↓

```
affine.for %arg2' = 0 to 10 {  
  affine.for %arg2 = (d0) -> (d0)(%arg2') to (d0) -> (d0 + 1)(%arg2') {  
    affine.store %cst, %0[%arg2] : memref<10xf32>  
    affine.store %cst, %1[%arg2] : memref<10xf32>  
  }  
  %2 = affine.load %1[%arg2'] : memref<10xf32>  
  %3 = arith.mulf %2, %2 : f32  
  affine.store %3, %arg1[%arg2'] : memref<10xf32>  
}
```

⇒

```
affine.for %arg2 = 0 to 10 {  
  affine.store %cst, %0[%arg2] : memref<10xf32>  
  affine.store %cst, %1[%arg2] : memref<10xf32>  
  %2 = affine.load %1[%arg2] : memref<10xf32>  
  %3 = arith.mulf %2, %2 : f32  
  affine.store %3, %arg1[%arg2] : memref<10xf32>  
}
```

fuseProducerConsumerNodes()

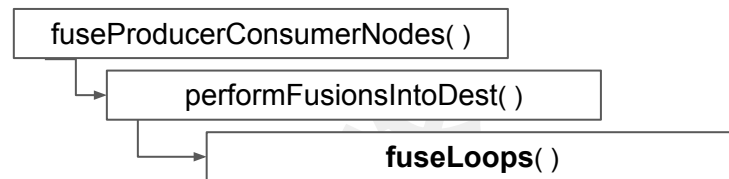
performFusionsIntoDest()

fuseLoops()

切片状态

```
ivs : { %arg2 }  
lbs : { (d0) -> (d0) }  
lbOperands : { %arg2' }  
ubs : { (d0) -> (d0 + 1) }  
ubOperands : { %arg2' }
```

执行循环融合



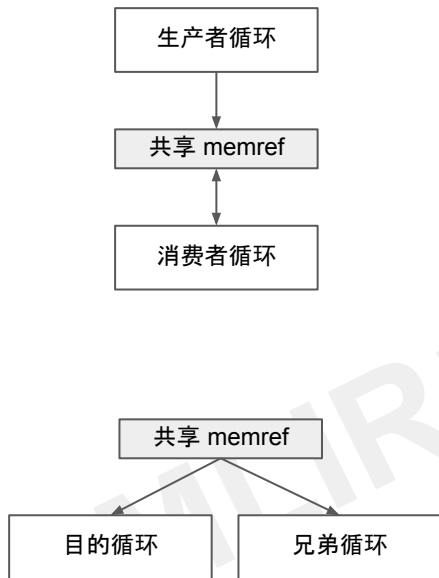
func.func @producer_consumer_fusion(%arg0: memref<10xf32>,
%arg1: memref<10xf32>) {
 %alloc = memref.alloc() : memref<1xf32>
 %alloc_0 = memref.alloc() : memref<10xf32>
 %cst = arith.constant 0.000000e+00 : f32
 affine.for %arg2 = 0 to 10 {
 affine.store %cst, %alloc_0[%arg2] : memref<10xf32>
 affine.store %cst, %alloc[0] : memref<1xf32>
 %0 = affine.load %alloc[0] : memref<1xf32>
 %1 = arith.mulf %0, %0 : f32
 affine.store %1, %arg1[%arg2] : memref<10xf32>
 }
 affine.for %arg2 = 0 to 10 {
 %0 = affine.load %alloc_0[%arg2] : memref<10xf32>
 %1 = arith.addf %0, %0 : f32
 affine.store %1, %arg0[%arg2] : memref<10xf32>
 }
 return
}

循环融合



func.func @producer_consumer_fusion(%arg0: memref<10xf32>,
%arg1: memref<10xf32>) {
 %alloc = memref.alloc() : memref<1xf32>
 %alloc_0 = memref.alloc() : memref<10xf32>
 %cst = arith.constant 0.000000e+00 : f32
 affine.for %arg2 = 0 to 10 {
 affine.store %cst, %alloc[0] : memref<1xf32>
 affine.store %cst, %alloc_0[0] : memref<1xf32>
 %0 = affine.load %alloc_0[0] : memref<1xf32>
 %1 = arith.mulf %0, %0 : f32
 affine.store %1, %arg1[%arg2] : memref<10xf32>
 %2 = affine.load %alloc_0[%arg2] : memref<1xf32>
 %3 = arith.addf %2, %2 : f32
 affine.store %3, %arg0[%arg2] : memref<10xf32>
 }
 return
}

兄弟融合测试用例



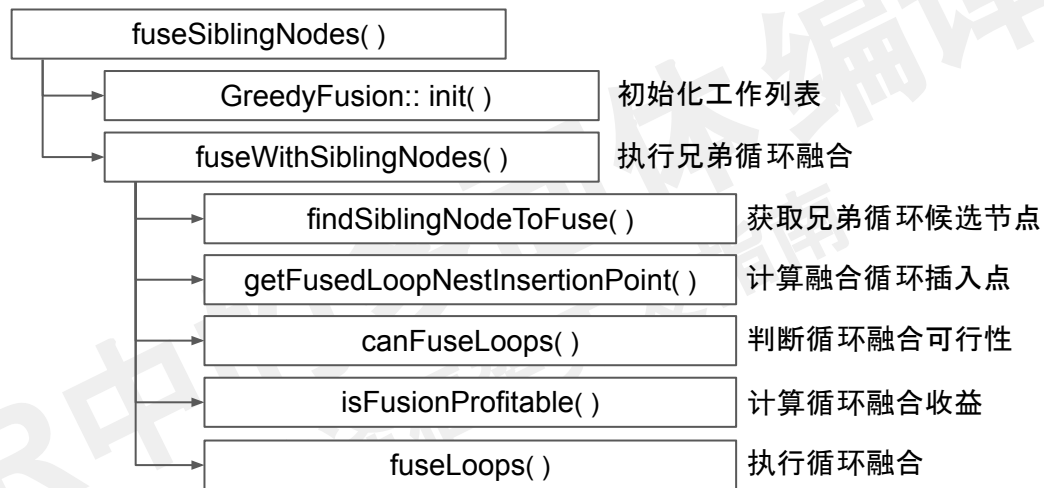
```
func.func @sibling_fusion(%arg0: memref<10x10xf32>, %arg1: memref<10x10xf32>,  
    %arg2: memref<10x10xf32>, %arg3: memref<10x10xf32>,  
    %arg4: memref<10x10xf32>) {  
  affine.for %arg5 = 0 to 3 {  
    affine.for %arg6 = 0 to 3 {  
      %0 = affine.load %arg0[%arg5, %arg6] : memref<10x10xf32>  
      %1 = affine.load %arg1[%arg5, %arg6] : memref<10x10xf32>  
      %2 = arith.mulf %0, %1 : f32  
      affine.store %2, %arg3[%arg5, %arg6] : memref<10x10xf32>  
    }  
  }  
  affine.for %arg5 = 0 to 3 {  
    affine.for %arg6 = 0 to 3 {  
      %0 = affine.load %arg0[%arg5, %arg6] : memref<10x10xf32>  
      %1 = affine.load %arg2[%arg5, %arg6] : memref<10x10xf32>  
      %2 = arith.addf %0, %1 : f32  
      affine.store %2, %arg4[%arg5, %arg6] : memref<10x10xf32>  
    }  
  }  
  return  
}
```

执行兄弟融合

- (1) 构建内存依赖图
- (2) 执行生产者-消费者融合
- (3) 执行兄弟融合

fuseWithSiblingNodes() 函数功能

- 1) 获取候选兄弟循环节点
- 2) 计算融合循环插入点
- 3) 判断兄弟融合可行性
- 4) 计算循环融合收益
- 5) 执行循环融合



判断兄弟融合可行性

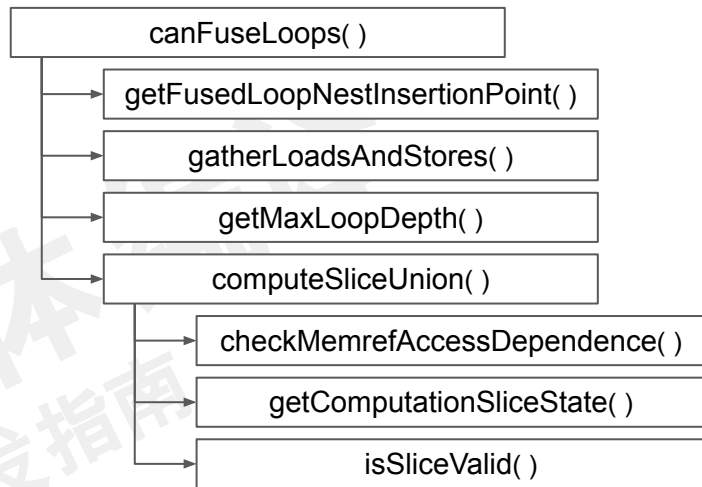
```
func.func @sibling_fusion(%arg0: memref<10x10xf32>, %arg1: memref<10x10xf32>,  
  %arg2: memref<10x10xf32>, %arg3: memref<10x10xf32>, %arg4:  
  memref<10x10xf32>) {  
  affine.for %arg5 = 0 to 3 {  
    affine.for %arg6 = 0 to 3 {  
      %0 = affine.load %arg0[%arg5, %arg6] : memref<10x10xf32>  
      %1 = affine.load %arg1[%arg5, %arg6] : memref<10x10xf32>  
      %2 = arith.mulf %0, %1 : f32  
      affine.store %2, %arg3[%arg5, %arg6] : memref<10x10xf32>  
    }  
  }  
  affine.for %arg5 = 0 to 3 {  
    affine.for %arg6 = 0 to 3 {  
      %0 = affine.load %arg0[%arg5, %arg6] : memref<10x10xf32>  
      %1 = affine.load %arg2[%arg5, %arg6] : memref<10x10xf32>  
      %2 = arith.addf %0, %1 : f32  
      affine.store %2, %arg4[%arg5, %arg6] : memref<10x10xf32>  
    }  
  }  
  return  
}
```

候选兄弟循环

插入点

目的循环

兄弟融合的目标 memref



兄弟融合模式下 opsA 集合只收集 affine.load 操作

opsA

%0 = affine.load %arg0[%arg5, %arg6] : memref<10x10xf32>

opsB

%0 = affine.load %arg0[%arg5, %arg6] : memref<10x10xf32>

%1 = affine.load %arg2[%arg5, %arg6] : memref<10x10xf32>

affine.store %2, %arg4[%arg5, %arg6] : memref<10x10xf32>

判断兄弟融合可行性

checkMemrefAccessDependence()

```
affine.for %arg5 = 0 to 3 {
  affine.for %arg6 = 0 to 3 {
    %0 = affine.load %arg0[%arg5, %arg6] :
    %1 = affine.load %arg1[%arg5, %arg6] :
    ...
  }
  affine.for %arg5 = 0 to 3 {
    affine.for %arg6 = 0 to 3 {
      %0 = affine.load %arg0[%arg5, %arg6] :
      %1 = affine.load %arg1[%arg5, %arg6] :
      ...
    }
  }
```



dependenceConstraints

```
(-1) * %arg5 + (0) * %arg6 + (1) * %arg5' + (0) * %arg6' + 0 = 0
(0) * %arg5 + (-1) * %arg6 + (0) * %arg5' + (1) * %arg6' + 0 = 0
(1) * %arg5 + (0) * %arg6 + (0) * %arg5' + (0) * %arg6' + 0 >= 0
(-1) * %arg5 + (0) * %arg6 + (0) * %arg5' + (0) * %arg6' + 2 >= 0
(0) * %arg5 + (1) * %arg6 + (0) * %arg5' + (0) * %arg6' + 0 >= 0
(0) * %arg5 + (-1) * %arg6 + (0) * %arg5' + (0) * %arg6' + 2 >= 0
(0) * %arg5 + (0) * %arg6 + (1) * %arg5' + (0) * %arg6' + 0 >= 0
(0) * %arg5 + (0) * %arg6 + (-1) * %arg5' + (0) * %arg6' + 2 >= 0
(0) * %arg5 + (0) * %arg6 + (0) * %arg5' + (1) * %arg6' + 0 >= 0
(0) * %arg5 + (0) * %arg6 + (0) * %arg5' + (-1) * %arg6' + 2 >= 0
```

getComputationSliceState()



sliceState/sliceUnion

```
ivs:{%arg5, %arg6}
lbs:{{(d0, d1) -> (d0), (d0, d1) -> (d1)}}
lbOperands:{%arg5', %arg6'}
ubs:{{(d0, d1) -> (d0 + 1), (d0, d1) -> (d1 + 1)}}
ubOperands : {%arg5', %arg6'}
```



```
affine.for %arg5 = 0 to 3 {
  affine.for %arg6 = 0 to 3 {
    %0 = affine.load %arg0[%arg5, %arg6] :
    ...
  }
  affine.for %arg5 = 0 to 3 {
    affine.for %arg6 = 0 to 3 {
      %0 = affine.load %arg0[%arg5, %arg6] :
      ...
    }
  }
```

计算切片为最大切片

isSliceValid()

isSliceMaximalFastCheck()

canFuseLoops()

getFusedLoopNestInsertionPoint()

gatherLoadsAndStores()

getMaxLoopDepth()

computeSliceUnion()

checkMemrefAccessDependence()

getComputationSliceState()

isSliceValid()

```
affine.for %arg5' = 0 to 3 {
  affine.for %arg6' = 0 to 3 {
    affine.for %arg5 = (d0, d1) -> (d0)(%arg5', %arg6') to
      (d0, d1) -> (d0 + 1)(%arg5', %arg6') {
      affine.for %arg6 = (d0, d1) -> (d1)(%arg5', %arg6') to
        (d0, d1) -> (d1 + 1)(%arg5', %arg6') {
        %0 = affine.load %arg0[%arg5, %arg6] : memref<10x10xf32>
        %1 = affine.load %arg1[%arg5, %arg6] : memref<10x10xf32>
        %2 = arith.mulf %0, %1 : f32
        affine.store %2, %arg3[%arg5, %arg6] : memref<10x10xf32>
      }
    }
    %3 = affine.load %arg0[%arg5', %arg6'] : memref<10x10xf32>
    %4 = affine.load %arg2[%arg5', %arg6'] : memref<10x10xf32>
    %5 = arith.addf %3, %4 : f32
    affine.store %5, %arg4[%arg5', %arg6'] : memref<10x10xf32>
  }
}
```

计算循环融合收益



计算成本 = 操作数量 × 循环迭代总数

additionalComputeFraction =

$$\text{fusedLoopNestComputeCost} / (\text{srcLoopNestCost} + \text{dstLoopNestCost}) - 1 \\ = 0 < 30\% (\text{computeToleranceThreshold})$$

$$\text{srcMemSizeVal} + \text{dstMemSizeVal} = 108 + 108 = 216$$

$$\text{fusedMem} = \text{dstMemSizeVal} + \text{sliceMemEstimate} = 108 + 36 = 144 < 216$$

```
affine.for %arg5 = 0 to 3 {
  affine.for %arg6 = 0 to 3 {
    %0 = affine.load %arg0[%arg5, %arg6] : memref<10x10xf32>
    %1 = affine.load %arg1[%arg5, %arg6] : memref<10x10xf32>
    %2 = arith.mulf %0, %1 : f32
    affine.store %2, %arg3[%arg5, %arg6] : memref<10x10xf32>
  }
}
```

$$\text{srcLoopNestCost} = 4 \times 9 = 36$$

$$\text{sliceMemEstimate} = 4 \times 9 = 36 \text{ bytes}$$

$$\text{srcMemSizeVal} = 4 \times 9 \times 3 = 108 \text{ bytes}$$

```
affine.for %arg5 = 0 to 3 {
  affine.for %arg6 = 0 to 3 {
    %0 = affine.load %arg0[%arg5, %arg6] : memref<10x10xf32>
    %1 = affine.load %arg2[%arg5, %arg6] : memref<10x10xf32>
    %2 = arith.addf %0, %1 : f32
    affine.store %2, %arg4[%arg5, %arg6] : memref<10x10xf32>
  }
}
```

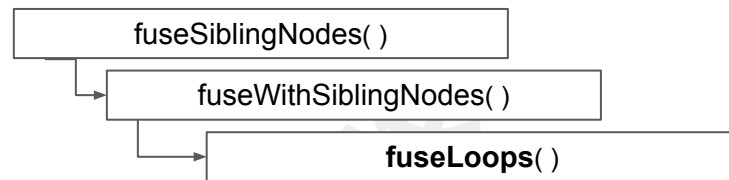
$$\text{dstLoopNestCost} = 4 \times 9 = 36$$

$$\text{dstMemSizeVal} = 4 \times 9 \times 3 = 108 \text{ bytes}$$

```
affine.for %arg5' = 0 to 3 {
  affine.for %arg6' = 0 to 3 {
    affine.for %arg5 = (d0, d1) -> (d0)(%arg5', %arg6') to (d0, d1) -> (d0 + 1)(%arg5', %arg6') {
      affine.for %arg6 = (d0, d1) -> (d1)(%arg5', %arg6') to (d0, d1) -> (d1 + 1)(%arg5', %arg6') {
        %0 = affine.load %arg0[%arg5, %arg6] : memref<10x10xf32>
        %1 = affine.load %arg1[%arg5, %arg6] : memref<10x10xf32>
        %2 = arith.mulf %0, %1 : f32
        affine.store %2, %arg3[%arg5, %arg6] : memref<10x10xf32>
      }
    }
    %3 = affine.load %arg0[%arg5', %arg6'] : memref<10x10xf32>
    %4 = affine.load %arg2[%arg5', %arg6'] : memref<10x10xf32>
    %5 = arith.addf %3, %4 : f32
    affine.store %5, %arg4[%arg5', %arg6'] : memref<10x10xf32>
  }
}
```

$$\text{fusedLoopNestComputeCost} = 8 \times 9 = 72$$

执行循环化融合



```
affine.for %arg5 = 0 to 3 {  
  affine.for %arg6 = 0 to 3 {  
    %0 = affine.load %%arg0[%arg5, %arg6] : memref<10x10xf32>  
    %1 = affine.load %arg1[%arg5, %arg6] : memref<10x10xf32>  
    %2 = arith.mulf %0, %1 : f32  
    affine.store %2, %arg3[%arg5, %arg6] : memref<10x10xf32>  
  }  
}  
affine.for %arg5 = 0 to 3 {  
  affine.for %arg6 = 0 to 3 {  
    %0 = affine.load %%arg0[%arg5, %arg6] : memref<10x10xf32>  
    %1 = affine.load %arg2[%arg5, %arg6] : memref<10x10xf32>  
    %2 = arith.addf %0, %1 : f32  
    affine.store %2, %arg4[%arg5, %arg6] : memref<10x10xf32>  
  }  
}
```



```
affine.for %arg5 = affine_map<(d0) -> (d0)>(%arg5') to 3 {  
  affine.for %arg6 = 0 to 3 {  
    %0 = affine.load %arg0[%arg5, %arg6] : memref<10x10xf32>  
    %1 = affine.load %arg1[%arg5, %arg6] : memref<10x10xf32>  
    %2 = arith.mulf %0, %1 : f32  
    affine.store %2, %arg3[%arg5, %arg6] : memref<10x10xf32>  
  }  
}
```

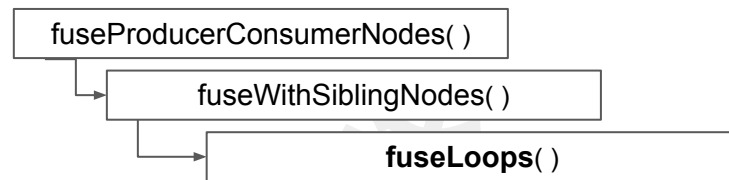


```
affine.for %arg5 = affine_map<(d0) -> (d0)>(%arg5') to affine_map<(d0) -> (d0 + 1)>(%arg5') {  
  affine.for %arg6 = 0 to 3 {  
    %0 = affine.load %arg0[%arg5, %arg6] : memref<10x10xf32>  
    %1 = affine.load %arg1[%arg5, %arg6] : memref<10x10xf32>  
    %2 = arith.mulf %0, %1 : f32  
    affine.store %2, %arg3[%arg5, %arg6] : memref<10x10xf32>  
  }  
}
```

切片状态

```
ivs:{%arg5, %arg6}  
lbs:{{(d0, d1) -> (d0), (d0, d1) -> (d1)}}  
lbOperands:{%arg5', %arg6'}  
ubs:{{(d0, d1) -> (d0 + 1), (d0, d1) -> (d1 + 1)}}  
ubOperands : {%arg5', %arg6'}
```

执行循环化融合



```
affine.for %arg5 = 0 to 3 {  
  affine.for %arg6 = 0 to 3 {  
    %0 = affine.load %arg0[%arg5, %arg6] : memref<10x10xf32>  
    %1 = affine.load %arg1[%arg5, %arg6] : memref<10x10xf32>  
    %2 = arith.mulf %0, %1 : f32  
    affine.store %2, %arg3[%arg5, %arg6] : memref<10x10xf32>  
  }  
}
```



```
affine.for %arg5 = 0 to 3 {  
  affine.for %arg6 = 0 to 3 {  
    %0 = affine.load %arg0[%arg5, %arg6] : memref<10x10xf32>  
    %1 = affine.load %arg2[%arg5, %arg6] : memref<10x10xf32>  
    %2 = arith.addf %0, %1 : f32  
    affine.store %2, %arg4[%arg5, %arg6] : memref<10x10xf32>  
  }  
}
```

切片状态

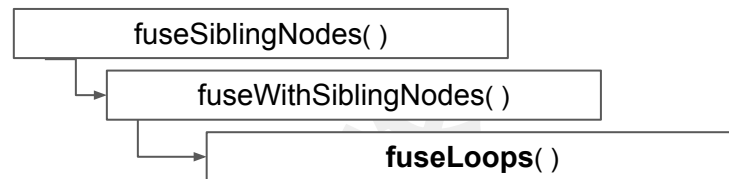
```
ivs:{%arg5, %arg6}  
lbs:{{(d0, d1) -> (d0), (d0, d1) -> (d1)}}  
lbOperands:{%arg5', %arg6'}  
ubs:{{(d0, d1) -> (d0 + 1), (d0, d1) -> (d1 + 1)}}  
ubOperands : {%arg5', %arg6'}
```

```
affine.for %arg5 = affine_map<(d0) -> (d0)>(%arg5') to affine_map<(d0) -> (d0 + 1)>(%arg5') {  
  affine.for %arg6 = affine_map<(d0) -> (d0)>(%arg6') to 3 {  
    %0 = affine.load %arg0[%arg5, %arg6] : memref<10x10xf32>  
    %1 = affine.load %arg1[%arg5, %arg6] : memref<10x10xf32>  
    %2 = arith.mulf %0, %1 : f32  
    affine.store %2, %arg3[%arg5, %arg6] : memref<10x10xf32>  
  }  
}
```



```
affine.for %arg5 = affine_map<(d0) -> (d0)>(%arg5') to affine_map<(d0) -> (d0 + 1)>(%arg5') {  
  affine.for %arg6 = affine_map<(d0) -> (d0)>(%arg6') to affine_map<(d0) -> (d0 + 1)>(%arg6') {  
    %0 = affine.load %arg0[%arg5, %arg6] : memref<10x10xf32>  
    %1 = affine.load %arg1[%arg5, %arg6] : memref<10x10xf32>  
    %2 = arith.mulf %0, %1 : f32  
    affine.store %2, %arg3[%arg5, %arg6] : memref<10x10xf32>  
  }  
}
```

执行循环化融合



```
affine.for %arg5 = 0 to 3 {  
  affine.for %arg6 = 0 to 3 {  
    %0 = affine.load %arg0[%arg5, %arg6] : memref<10x10xf32>  
    %1 = affine.load %arg1[%arg5, %arg6] : memref<10x10xf32>  
    %2 = arith.mulf %0, %1 : f32  
    affine.store %2, %arg3[%arg5, %arg6] : memref<10x10xf32>  
  }  
}  
affine.for %arg5 = 0 to 3 {  
  affine.for %arg6 = 0 to 3 {  
    %0 = affine.load %arg0[%arg5, %arg6] : memref<10x10xf32>  
    %1 = affine.load %arg2[%arg5, %arg6] : memref<10x10xf32>  
    %2 = arith.addf %0, %1 : f32  
    affine.store %2, %arg4[%arg5, %arg6] : memref<10x10xf32>  
  }  
}
```



```
affine.for %arg5 = 0 to 3 {  
  affine.for %arg6 = 0 to 3 {  
    %0 = affine.load %arg0[%arg5, %arg6] : memref<10x10xf32>  
    %1 = affine.load %arg1[%arg5, %arg6] : memref<10x10xf32>  
    %2 = arith.mulf %0, %1 : f32  
    affine.store %2, %arg3[%arg5, %arg6] : memref<10x10xf32>  
    %3 = affine.load %arg0[%arg5, %arg6] : memref<10x10xf32>  
    %4 = affine.load %arg2[%arg5, %arg6] : memref<10x10xf32>  
    %5 = arith.addf %3, %4 : f32  
    affine.store %5, %arg4[%arg5, %arg6] : memref<10x10xf32>  
  }  
}
```